



Boosting Symbolic Execution via Constraint Solving Time Prediction (Experience Paper)

Sicheng Luo

Hui Xu*

Yanxiang Bi

School of Computer Science, Fudan University
China

Xin Wang

Yangfan Zhou

School of Computer Science, Fudan University
Shanghai Key Lab of Intelligent Information Processing
China

ABSTRACT

Symbolic execution is an essential approach for automated test case generation. However, the approach is generally not scalable to large programs. One critical reason is that the constraint solving problems in symbolic execution are generally hard. Consequently, the symbolic execution process may get stuck in solving such hard problems. To mitigate this issue, symbolic execution tools generally rely on a timeout threshold to terminate the solving. Such a timeout is generally set to a fixed, predefined value, e.g., five minutes in angr. Nevertheless, how to set a proper timeout is critical to the tool's efficiency. This paper proposes an approach to tackle the problem by predicting the time required for solving a constraint model so that the symbolic execution engine could base on the information to determine whether to continue the current solving process. Due to the cost of the prediction itself, our approach triggers the predictor only when the solving time has exceeded a relatively small value. We have shown that such a predictor can achieve promising performance with several different machine learning models and datasets. By further employing an adaptive design, the predictor can achieve an F1-score ranging from 0.743 to 0.800 on these datasets. We then apply the predictor to eight programs and conduct simulation experiments. Results show that the efficiency of constraint solving for symbolic execution can be improved by 1.25x to 3x, depending on the distribution of the hardness of their constraint models.

CCS CONCEPTS

• **Software and its engineering** → **Software testing and debugging**.

KEYWORDS

Symbolic execution, SMT solving, Adaptive machine learning

ACM Reference Format:

Sicheng Luo, Hui Xu, Yanxiang Bi, Xin Wang, and Yangfan Zhou. 2021. Boosting Symbolic Execution via Constraint Solving Time Prediction (Experience Paper). In *Proceedings of the 30th ACM SIGSOFT International Symposium on*

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ISSTA '21, July 11–17, 2021, Virtual, Denmark

© 2021 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-8459-9/21/07...\$15.00

<https://doi.org/10.1145/3460319.3464813>

Software Testing and Analysis (ISSTA '21), July 11–17, 2021, Virtual, Denmark.
ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3460319.3464813>

1 INTRODUCTION

Symbolic execution is an automated test case generation technique widely used for software testing and dynamic program analysis. It is in nature a white-box approach that accumulates path conditions along an execution path into a constraint model. By solving the constraint model with a constraint solver, the symbolic execution tool can find the test input for triggering certain paths and errors.

Nevertheless, symbolic execution tools (e.g., KLEE [10], angr [42], and Manticore [33]) generally suffer from obvious limitations [53]. For instance, some code snippets involve constraints of challenging theories, such as floating-point arithmetic [28, 30, 38] and arrays [36, 37]. Furthermore, symbolic execution has poor scalability in handling large programs because the number and complexity of paths grow exponentially as the depth of the control-flow graph increases, and some paths inevitably involve complex constraint models that require much time to solve. It may take a few days to explore all available paths for a challenging program, and constraint solvers could drain a considerable amount of the resources allocated for symbolic execution [34]. In short, the cost of constraint solving is an essential obstacle for symbolic execution towards being more practical in analyzing real-world programs.

Currently, most symbolic execution tools employ a timeout threshold to automatically terminate the constraint solving process and avoid unrestrained time growth. For example, the default threshold of angr for constraint solving is five minutes. If a solving process exceeds the time limit, angr would give up exploring the path and pick up a new path for analysis. As a result, angr would accumulatively waste a large amount of time on the paths that would be eventually given up. The effectiveness of solving largely depends on the choice of the threshold and the number of constraint models that exceed the time limit.

This paper focuses on saving the futile effort of solving constraint models that cannot be solved within the timeout threshold. Intuitively, if a symbolic execution tool could predict the solving time, it may hang up the unavailing constraint solving process or assign a low priority to the path so that path with an easier constraint would be explored first. Given the fact that real-world software often contains millions of or even an infinite number of paths, solving simple paths first will definitely speed up the solving to achieve a higher path coverage within a limited time budget.

To this end, our work proposes to tackle the constraint solving time inference problem as a machine learning problem. The input to our problem is a constraint model of SMT-LIB format, which

is the most widely employed format by symbolic execution tools, including angr, KLEE, and Manticore. The answer to our problem can be either a specific time required to solve the problem or the solvability within a time threshold. Since the problem-solving ability varies a lot among different constraint solvers [51], our work does not intend to predict the answer for general solvers. Our model targets a particular solver, *i.e.*, Z3 [12], a state-of-the-art SMT solver adopted by all the previously mentioned symbolic execution tools.

This paper studies the practicability of the time prediction with two different types of approaches, non-adaptive ones, and adaptive ones. For non-adaptive approaches, we employ both traditional feature-based approaches (*e.g.*, KNN) and recently emerged neural networks (*e.g.*, LSTM) for code learning [39] because the constraint models of SMT-LIB are in nature a code representation. Since the problem is complicated, such non-adaptive models trained with limited data can hardly be generalized to new programs. Therefore, we also propose an adaptive approach to solve the problem based on online learning [24]. Our approach, named incremental KNN, continuously updates the KNN model during runtime with the constraint models generated by the current program under symbolic execution. The underlying assumption is that the constraint models generated by the same program should share more similarity in syntactic structure and thus can be more helpful for prediction.

To evaluate the effectiveness of our approach, we conduct experiments on four datasets. Three of them are generated from angr and KLEE with GNU and BusyBox program datasets, and the rest is from the benchmark dataset of SMT-COMP. Our collected datasets contain more than 200,000 constraint models from over 200 programs. For each dataset, we partition the datasets with the program name to ensure that the training data and testing data are independent. Experimental results show that the F1-score of non-adaptive approaches ranges from 0.2 to 0.75 for solvability prediction on these datasets, while the adaptive approach can achieve a stable F1-score over 0.74. We further simulate the solving process on the test programs. Experimental results show that the constraint solving is generally improved than the original solving, ranging from 1.25x to 3x depending on the proportion of timeout cases. We summarize the main contributions of this paper as follows.

- This paper is a pilot study for improving the efficiency of symbolic execution by predicting the solving time of specific constraint models. We propose a systematic framework that employs the solving time prediction to guide the process of constraint solving. Experimental results show that our approach can improve the efficiency of symbolic execution.
- We employ both non-adaptive and adaptive approaches to infer the constraint solving time. Our approaches outperform the baseline PCC (a DNN classification with well-defined features) [51] in most cases, where our adaptive approach performs the best compared with the non-adaptive approaches and achieves the F1-score of 0.74.
- We have collected a large volume of constraint models of SMT-LIB format containing some hard-solving problems. These data are useful for other researchers to investigate the solving time prediction problem. We share our source code and data on <https://github.com/Artisan-Lab/SMTTimer> with the community for conducting further studies.

2 BACKGROUND

2.1 Challenges of Symbolic Execution

Symbolic execution is a program analysis technique that can generate inputs for achieving high code coverage [11, 26]. The traditional symbolic execution technique simulates the running process. It does not use concrete values when program inputs are required. Instead, it symbolizes the inputs, and then the program variables can be represented as expressions of these symbolic inputs. It maintains the symbolic state of the path with a symbol map and a set of path constraints, which are first-order formulae over the variables of the symbol map. When the execution encounters a new conditional statement, it updates the path constraint set with a new constraint, the condition formula or its negation, to generate two different path states, which mean two paths to be explored. The constraint models are then sent to the solver to check their satisfiability and concretize these symbolic values. While the satisfiability of constraint models represents the reachability of corresponding paths, their concrete values imply the test cases that can trigger certain paths that have been symbolically executed. In this way, symbolic execution can be applied to test case generation and vulnerability detection.

However, due to the limitation of computing power, symbolic execution has not been developed rapidly until some research mix concrete and symbolic execution to alleviate the solving problem called concolic execution, *e.g.*, DART [20] and CUTE [40]. Concolic execution maintains the symbolic state to generate inputs for new paths while running with concrete values. Our work mainly focuses on angr since it is one of the best open-source symbolic execution tools in analyzing capability [53] and also uses concolic execution.

For now, symbolic execution tools can only show their full capability on small applications. Large applications usually start with some interesting inputs as seeds to explore some specific path analysis, because they have scalability issues in both path explosion and hard constraint solving. We discuss those two challenges in the next two paragraphs.

For each conditional statement, symbolic execution would generate two new states, but it can only choose one to continue the exploration. The path scale would grow rapidly if the current path cannot reach the exit point soon. Meanwhile, a for loop may create a large number of paths that cannot reach the exit point when the condition formula contains symbolic values. Therefore, users usually use symbolic execution to explore specific, interesting paths (*e.g.*, vulnerable code snippets). Otherwise, the path scale would often grow with time and require manual pruning or termination. That is why its most important evaluation metrics are instruction coverage and path coverage within a limited time.

The constraint solving in symbolic execution employs the solving theory of first-order logic. Both program structures and execution strategies would affect the formula complexity and its solving time. For example, the loops would cause a huge accumulation of repeated logical structures. Symbolic execution strategies (*e.g.*, merging states [27]) would put extra burdens on constraint solving with transferring the path complexity into constraint formula complexity. Besides, the reasoning theory and solving methods are other important factors of solving time.

Next, we briefly review the SMT solving problem and the typical format of SMT models.

```

1 #include <stdio.h>
2
3 int main(int argc, char* symvar[]) {
4     if (symvar[1][0] == 48){
5         printf("0");
6     }
7     else if (symvar[1][0] == 49){
8         printf("1");
9     }
10    return 0;
11 }

```

(a) A toy example with two branching points.

```

1 (declare-fun sym_argv_0_80 () (_ BitVec 80))
2 (assert (let ((?x10 ((_ extract 79 72) sym_argv_0_80)))
3     (= (_ bv48 8) ?x10))) (check-sat)
4 (declare-fun sym_argv_0_80 () (_ BitVec 80))
5 (assert (let ((?x10 ((_ extract 79 72) sym_argv_0_80)))
6     (and (distinct (_ bv48 8) ?x10) true)))
7 (assert (let ((?x10 ((_ extract 79 72) sym_argv_0_80)))
8     (and (distinct (_ bv49 8) ?x10) true))) (check-sat)
9 (declare-fun sym_argv_0_80 () (_ BitVec 80))
10 (assert (let ((?x10 ((_ extract 79 72) sym_argv_0_80)))
11     (and (distinct (_ bv48 8) ?x10) true)))
12 (assert (let ((?x10 ((_ extract 79 72) sym_argv_0_80)))
13     (= (_ bv49 8) ?x10))) (check-sat)

```

(b) Three constraint models (1-3, 4-8, 9-13) of the program in Fig.1(a) written in SMT-LIB language.

Figure 1: A toy example and its three constraint model scripts.

2.2 SMT Solving

Satisfiability Modulo Theories (SMT) focuses on first-order logic solving, which is widely used in software analysis [5]. It is a generalization of the SAT problem [23] in which a variety of underlying theories expand the representation of SAT variables. It includes theories of arithmetical reasoning like linear and nonlinear arithmetic, program analysis oriented theories for arrays, bit-vectors, strings, and floating-point numbers, and their combinations.

For a constraint model, an SMT solver would check its satisfiability and look for solutions. To adapt to the complex and diverse constraint formulas generated by different programs, new solving approaches have been continuously proposed, trying to optimize the solution process or reuse constraint information. [10, 12] These approaches include pruning irrelevant constraints, simplifying complex arithmetic formulas, caching solutions for reusing, and using fuzz to search for complex constraints. Like reusing constraint information, our approach aims to selectively solve constraints based on experience. Instead of wasting time on timeout constraints, we want to spend time on the path with more possibilities to solve, to achieve higher path coverage in symbolic execution.

2.3 Format of SMT Script

To better explain our feature extraction, we introduce the constraint format first. The SMT-LIB language provides a standard to write down well-sorted first-order formulas [3, 4]. It is designed to unify the input language of SMT solvers, and many SMT solvers accept scripts of SMT-LIB standard as their input. The SMT scripts consist of commands that declare and define function symbols, add assertions, control assertion stack, and check satisfiability. Each command has a fixed number of arguments, which can be terms defined before.

There are three SMT scripts in Fig. 1(b) generated from two conditional statements of the program in Fig. 1(a). A standard SMT script uses the `declare-fun` command to declare variables (e.g., `sym_argv_0_80`) and functions with its data type and length. Then each `assert` command adds an assertion into the assertion stack. Every assertion is a logical formula, and the assertion stack is a conjunction of these formulas. The first SMT script has only one assertion, while the last two have two assertions. Unlike our examples,

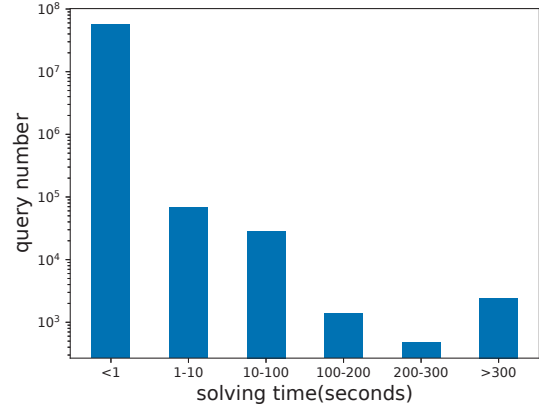


Figure 2: Distribution of constraint solving time for GNU Coreutils programs.

the assertion stack generated from real applications often contains hundreds of assertions, and the reasoning process of assertions would be time-consuming. The constraint solving time is related to the scale of assertion and computation. Therefore, we can infer the solving time from the statistical features of the constraint model. In the end, the `check-sat` command triggers the satisfiability check.

3 PROBLEM DESCRIPTION

In this section, we first present our motivation with the distribution of constraint solving time that has been collected from a symbolic execution experiment. Then we propose a promising system architecture for dealing with the problem. Based on the architecture, we formulate the solving time optimization problem that we aim to solve in this work.

3.1 Motivation

Our work is motivated by the phenomenon that symbolic execution tools generally spend a considerable amount of time during

constraint solving and some of them are meaningless. This part presents a case study showing how much time would be wasted.

Our experiment runs symbolic execution with angr on the program set of GNU Coreutils [10], which contains 102 different programs. We conduct a 12 hours experiment on every program. Only 22 programs have completed the symbolic execution process within this time limit, while the other 80 programs have not. Then we record all the constraint solving time spent during the experiments to analyze their distributions.

Fig. 2 shows the distribution of the constraint solving time. Overall, there are over 50 million constraint models generated during our experiment, and 99.8% of them have been solved within one second. However, the problem lies in the remaining 0.2%. In particular, there are thousands of constraint models that have not been solved, and angr terminates the solving process, also path exploration, once it reaches the default time limit, *i.e.*, 300 seconds. Although the proportion of these cases is tiny within all constraint models, when multiplied with the time base (*e.g.*, 300 seconds), nearly half of the constraint solving time has been wasted on such meaningless solving. Moreover, the total time spent on constraint solving accounts for half of all the symbolic execution time during our experiment (*i.e.*, 498 hours in 960 hours). Therefore, if a symbolic execution tool can know the required solving time in advance, it may save the wasted time for exploring other pending paths.

3.2 Architecture

To avoid the unavailing constraint solving process in symbolic execution, we design SMTimer for forecasting the needed time for constraint solving. Fig. 3 shows how our system utilizes our predicted time and interacts with a running symbolic execution tool. Our prediction model takes a constraint model generated from an on-going symbolic execution program as input, and outputs the constraint solvability. Then we go back to the SMT solver or help the symbolic execution tool determine how to process the path.

One specific design of SMTimer is that the prediction process is triggered conditionally. The reason is that if we predict every time it queries, even for a short prediction cost, we would add unbearable overhead considering the massive number of queries. Considering that only 0.2% of queries would use up to one second, so we can solve the constraint model with a short time first to filter out easy constraint models. By spending a little extra solving time, we avoid most predictions. This method also ensures that our prediction module does not bring in any side effects for programs only having fast-solving queries.

To better explain how the system works, we discuss more details about the key steps of the new constraint solving process below.

- **Solving Phase 1:** Similar to the traditional constraint solving process, the system first tries to solve the constraint model within a time limit t_1 (*e.g.*, one second). If it can successfully output a solution, there is no difference compared to classic symbolic execution. If it cannot solve the constraint model within t_1 , the prediction is triggered.
- **Prediction:** The system uses a predictor to forecast the solving time and compare it with the time threshold t_2 . If the predicted time is larger than t_2 , it would skip the next solving phase and feedback an unknown result to the symbolic

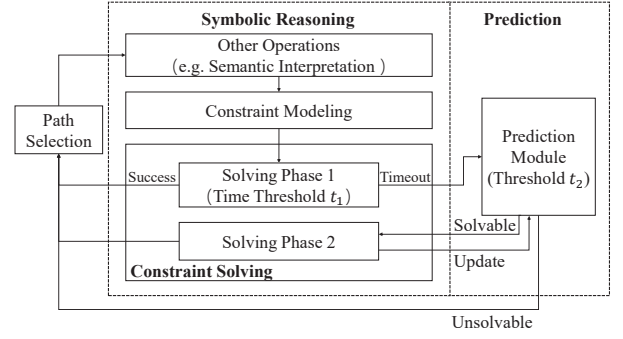


Figure 3: The framework of our system.

execution tool. Otherwise, it would go back to constraint solving for the second solving phase.

- **Solving Phase 2:** The system solves the constraint model with the original time limit. Then constraint model with the ground truth would update the adaptive model to optimize the predictor.

We use this workflow in our simulation experiment to check the effectiveness of our prediction and show the improvement in the time optimization problem we defined in the next part.

3.3 Problem Definition

We now define the solving time optimization problem in symbolic execution. For a sequence of constraint models $PC = \{pc_1, \dots, pc_n\}$, the original time consumption can be represented as Eq. (1).

$$T = \sum_i^n (S_i + O_i) \quad (1)$$

where S_i denotes time spent on constraint solving of pc_i and O_i denotes the time for other operations.

In the new system, the time consumption T' can be represented as Eq. (2) with two parts, time spent on symbolic execution for solvable paths and our prediction overhead.

$$T' = \sum_{S_i < t_1} (S_i + O_i) + \sum_{S_i > t_1, R_i < t_2} (S_i + O_i) + \sum_{S_i > t_1} (P_i + t_1) \quad (2)$$

R_i denotes our predicted solving time, and P_i denotes the time consumption of prediction. The time threshold t_1 is used to trigger the prediction and the time threshold t_2 is used to decide whether to solve the constraint model with the original time limit.

In comparison, the time difference is represented as Eq.(3). Note that we cancel the time O_i spent on other symbolic execution operations because such time generally remains the same in our task.

$$T - T' = \sum_{R_i > t_2} S_i - \sum_{S_i > t_1} (P_i + t_1) \quad (3)$$

Next, we discuss how we choose the thresholds t_1 and t_2 , as well as their influence on the time difference. t_1 is determined by solving time distribution to achieve the least overhead and t_2 usually is the symbolic execution tool's time limit. But you can also take the value which achieves a better prediction result.

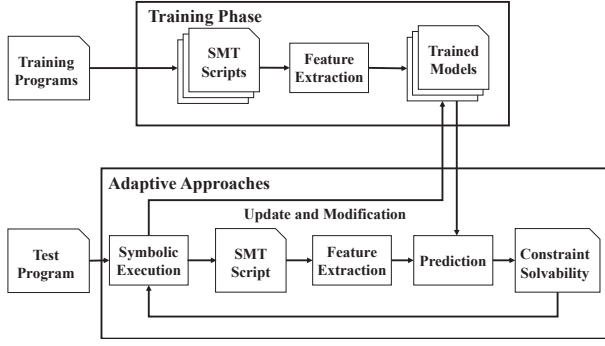


Figure 4: Approach for constraint solving time prediction.

When t_1 is set to one second, our system would take an extra 1.1 seconds for every constraint that has triggered the prediction. According to Eq.(3) and the time distribution, it would generate about a hundred thousand seconds of overhead at most. As long as 15% (i.e., 370 of 2438) of timeout queries correctly detected, we can boost the constraint solving.

However, some programs have no timeout constraint, then the first part of Eq.(3) $\sum_{R_i > t_2} S_i$ is zero. Our model cannot improve this kind of program, so we calculate the average degradation of performance, i.e., about 15 minutes within the 12 hours running, which is tolerable. The cost is mainly caused by the first solving phase, therefore, it can be minimized by finding suitable t_1 . Furthermore, when all the solving time locates within t_1 , no constraint models would trigger our prediction, then $\sum_{S_i > t_1} (P_i + t_1)$ would be zero too. Therefore, our system would not suffer performance degradation for programs without hard constraint models. Such a guarantee is essential considering the time distribution in Fig.2.

4 APPROACH FOR CONSTRAINT SOLVING TIME PREDICTION

This section presents both our non-adaptive and adaptive approaches for the solving time inference problem. Fig. 4 presents an overview of our prediction workflow, including three key steps, data collection, feature extraction, and modeling.

- **Data Collection:** We would keep receiving sequential constraint models of SMT-LIB format generated during symbolic execution. Besides, we have collected a large number of scripts for training.
- **Feature Extraction:** Feature extraction stems the constraint information from the SMT script and transforms each SMT script into the feature vectors needed.
- **Modeling:** We train different models to learn the relationship between SMT scripts and their solving time to predict solving time or solvability.

In the following paragraphs, we would describe the design of feature extraction and our approaches to predict solving time. The data collection would be illustrated in the next section.

4.1 Feature Extraction

Overall, the feature extraction parses the assertion stack of an SMT script into a two-dimensional feature vector. Every assertion in the

assertion stack would be transformed into a vector of 150 dimensions. The assertion, shown in Section 2.3, consists of operators, operands (a variable or a constant), and terms defined before. We tokenize the assertion to compute the feature number. Then we unify the feature size of the feature vector for the training and use the logarithm transformation to erase the skewness of the features [51].

SMT contains different operators for different theories. The arithmetic operators include add, sub, etc. The bit-vector operators include bvadd, bvand, bvor, etc. Besides, there are quantifiers such as forall and exists for logical inference. We use the symbol table in pySMT [19] to extract operators, which includes operators for the theories of bit-vector, linear integer arithmetic, and array. In particular, the logic operator and is served as the operator to conjunct assertions, which is used in scripts collected from KLEE and our Tree-LSTM feature structure. The result of the conjunctive normal form (CNF) corresponds to the satisfiability of the constraint model.

We count the number of operands for the symbolic variables and constants. Symbolic variables reflect the relation between assertions. The symbolic variables consist of the symbolic input of symbolic execution and the return values of unresolvable external functions. We rename the variables with $v_i (i = 1, 2, 3, \dots)$ to standardize constraint models. For constants, their number represents the computation amount of the current assertion. To reduce the feature dimension, we generalize all constants into bvconstant.

Based on our feature extraction method, each assertion generates a feature vector of 150 dimensions. However, a constraint model is composed of several or hundreds of assertions. To fit the features into our model, we unify the feature size. In detail, the number of feature vectors (150 dimensions) is 50 for LSTM and 2 for KNN, leading to the final feature size, i.e., $150 * 50$ for LSTM, and $150 * 2$ for KNN. For the constraint model with more assertions, we reserve the last $n - 1$ (e.g., 49 for LSTM) vectors of assertions and merge the excessive vectors of assertions by adding them into the first vector. The size unification method is based on our observation of the solving time difference between timeout constraint models and the last constraint before it along the same path. About 74%, i.e., 922 of 1252 of unsolvable constraint models' solving time is suddenly soaring from less than half a minute to 5 minutes when the last constraint is added. So we may assume the last constraint is more effective in solving time than other constraints.

4.2 Non-Adaptive Approaches

We use several machine learning methods, including neural networks and K-Nearest Neighbor (KNN), to forecast the solving time. Neural networks are one of the most popular approaches in the research of software analysis related texts and code recently. KNN is a widely-used non-parametric model.

4.2.1 Neural Networks. We use LSTM [31] and Tree-LSTM [43, 44], which have accomplished significant achievement in code analysis, to predict solving time. We choose these two neural network models because the constraint of SMT-LIB is a code representation. The code characteristics lie in the linear order of assertions and the hierarchy resulting from the order of computation.

First, we apply LSTM, a recursive neural network, to handle feature vectors sequentially. Similar to NLP (natural language processing) problems, the constraint model can be regarded as a sentence. Every assertion in the constraint model is like a word in the sentence. The word embedding reveals the semantic relation between words and reduces the feature dimension. Correspondingly, we extract the feature number from assertion as embedding to achieve a similar effect. In classification, the LSTM model would predict the solvability within a time threshold to help us skip unavailing solving. In regression, it could predict specific solving time to assign different priorities to the path.

Then, we employ the Tree-LSTM to make use of the hierarchy of mathematical reasoning. We have attempted to present the assertion as the expression tree [48] that the internal node represents the operator, and every subtree represents the result of under-layer reasoning. We give up the design since the improvement compared with the LSTM model is tiny, but the increased storage and calculation pressure are massive. For simplification, we use our extracted feature vector to represent the assertion. Then we merge the vector with the and operator, and get a similar representation of the expression tree. The input of Tree-LSTM is an expression tree of feature vectors and the output form is the same as the LSTM model.

4.2.2 K-Nearest Neighbor. We use KNN to classify solvability. In terms of the internal structure, our model uses Euclid distance to find the nearest constraints to vote the solvability. We choose three as the value of K for two reasons. First, our positive case number is relatively small. Therefore it would be hard to yield a positive result for a large K. Second, if K is large, the vectors that differ a lot are more likely to affect the final result. In our scenario, the diverse feature vectors mean the time difference could be huge. Therefore, we tend to use more similar vectors for prediction.

4.3 Adaptive Approach

We propose an adaptive approach that updates the predictor continuously during runtime. Because constraints are accumulated as the depth of an exploration path grows, the required solving time usually grows unless the solver detects conflicting clauses from the constraints. This relationship between sequential constraint and solving time encourages us to use constraints generated from the current program to update our model. We find similar thoughts of reusing constraint information in KLEE and Green. They both cache solutions to reduce queries. The time prediction does not involve theory reasoning, so our adaptive approach is easier but equally beneficial.

We have designed an adaptive KNN approach based on the concept of online learning [18, 46], named incremental KNN. KNN is a lazy learning method, which means all calculation processes and the time cost are deferred until the testing phase. Therefore, lazy learning makes it easy for us to adaptively increase data. We augment the training data with sequential constraint models generated during runtime to extend knowledge for the current program.

Specifically, our adaptive model interacts with symbolic execution tools to obtain new constraints from the current program. At the beginning of symbolic execution, the model loads the training data. Once a constraint solving triggers the predictor, our predictor would return constraint solvability. We use different strategies to

Table 1: Demonstration of our datasets. Raw data stands for the SMT scripts and solving time (second) directly collected from symbolic execution. After Adjustment stands for the solving time without symbolic execution.

dataset	Raw Data (s)		After Adjustment (s)	
	1-300	>300	1-300	>300
GNU coreutils (angr)	61,874	8,670	68,803	1,741
Busybox utilities (angr)	11,295	6,020	16,719	596
GNU coreutils (KLEE)	90,789	1,061	91,481	369
SMT-COMP	-	-	31,795	2,228

deal with different predicted results. If the result is solvable, the symbolic execution will query the constraint again to check satisfiability, which validates our forecasting result. If our predictor misjudges the solvability, the real solving result can correct our prediction. Therefore we have the ground truth of the current constraint to update our model. Then the constraint would be added to the training data. If the predicted result is unsolvable, we instruct symbolic execution to give up solving them and discard the data. Because the correctness of our predicted result is not examined, updating our model with the data may cause side effects in prediction. If these data keep increasing, the prediction may tend to bias towards undesired false positives. On the contrary, we prefer a negative result since its worst effect is almost equivalent to the original mechanism, wasting time in the SMT solver. In this way, all data including the increased ones are labeled with ground truth, which should be able to improve the prediction accuracy.

The model needs the initial data to avoid a cold start. The training data provide the ground truth to classify the solvability at the beginning of testing. As the test continues, the added data make up for unknown data patterns. This adaptive approach avoids the prediction result overfitting the training data.

5 EXPERIMENTAL STUDY

This section studies three research questions:

- RQ1: Can the machine learning techniques find the correlation between the solving time and constraint models?
- RQ2: Does the adaptive approach perform better than non-adaptive models?
- RQ3: Can the prediction improve the efficiency of constraint solving for single programs and what is the cost?

5.1 Experiment Setting

5.1.1 Raw Data Collection. To collect SMT scripts, we use both angr and KLEE to run symbolic execution experiments with two programs dataset, GNU Coreutils, a widely-used benchmark program set in symbolic execution research [10], and BusyBox, a collection of standard UNIX utilities. Take GNU Coreutils as an example, it contains 102 programs, which provide the basic utilities for the GNU operating system. We run each program for 12 hours with the solving timeout of 100 seconds to generate a considerable number of constraint models. Finally, we have collected over 70,000 constraint models of bit-vector theory with angr and over 90,000

constraint models of bit-vector and array theories with KLEE. We show more details of datasets in Table 1.

From the experiment in Section 3.1, there are millions of constraint models generated during symbolic execution. However, 99.8% of these models cost less than one second. We only record constraint models with solving time larger than one second. Since such data are much important in time consumption and we apply our approach only to these data in our research. Besides, recording all of them would require huge storage space. We divide our recorded data into two subsets. The solvable part contains 61,874 items, and it occupies 88% of the total dataset; the unsolvable subset contains 8,670 cases.

We also conduct experiments on a benchmark dataset from SMT-COMP [49], which contains SMT scripts following a comprehensive collection of solving theories. Unfortunately, the data sizes of other theories are small and may not be enough for training and evaluation, so our experiment mainly employs SMT scripts with bit-vector theory.

Constraint models in all datasets follow the SMT-LIB standard, which is the most popular SMT language used by symbolic execution tools (e.g., KLEE and angr). This ensures that although we only use angr and KLEE to collect data and z3 to solve constraints, our approach could apply to other symbolic execution tools and SMT solvers.

5.1.2 Label Adjustment and Dataset Selection. We observe that our captured SMT solving time during experiments may be inaccurate or unstable because it could be affected a lot by the runtime and contexts of symbolic execution, such as concurrent operations. To mitigate the influence, we further adjust the solving time by querying each model with a pure z3 solver outside symbolic execution. This can reduce the interference of other concurrent operations during symbolic execution.

After adjustment, the solving time of many models has changed. Specifically, most models require less time to find the solution, and a large number of the timeout constraint models become solvable. There are only about 5,000 timeout cases left in total. Considering the training data is heavily imbalanced, we conduct a random under-sampling [54] to reduce prediction bias.

In later experiments, we employ 2/3 of the programs for training and the remaining for testing to avoid the information leaks of the same program. Table 1 presents the detailed distribution of our raw data and data after adjustment.

5.1.3 Model Settings. Besides the adaptive module, we trained our models with the following settings.

The KNN model employs a window size K of three and Euclid distance to determine the similarity between different constraint models. By now, we use brutal search in this work.

For LSTM and Tree-LSTM, we set the hidden size of their layers to 150. We use AdaGrad [14] to optimize our model with the learning rate of 0.05, and each model is trained for 40 epochs. For LSTM, we use padding and summation to convert them to a fixed length of 50. For the Tree-LSTM model, we prune the tree to accelerate model training and testing.

5.2 Performance Evaluation of Our Predictor

In this part, we present the prediction results of different models for solvability and forecasting the constraint solving time. We compare the result with the baseline model PCC [51] to answer RQ1 and RQ2. PCC employs a fully connected neural network to select the best solver which implicitly contains the time consumption prediction task.

5.2.1 Classification of Timeout Constraint Models. To compare all the results fairly, we set the same time threshold t_2 , i.e., 200 seconds for all the models. We then evaluate the classification result with the metrics of precision, recall and F1-score [1]. A promising predictor should be able to achieve a good recall while maintaining high precision.

We prefer better precision over better recall. The preference is because false positives have a greater impact on symbolic execution. To elaborate, the false negative cases mean that we miss the constraint model that exceeds the time threshold we set. Then symbolic execution tools would query it with their preset threshold, but they would not be able to solve it. The solving time, as well as the extra cost of the first solving and the prediction overhead, are in vain. So more true positive cases found, more time wasted in unsolvable constraint models would be saved. On the other hand, the false positive cases mean that the predictor finds it is hard to solve the constraint model for the current time limit. Thus, you can postpone the paths by assigning them a low priority and explore them after other paths in symbolic execution. To explain, the solving phase is conducted at every conditional statement, and every finished path makes as many queries as the number of conditional statements. So the abandonment of the path does not necessarily mean the loss of code coverage, just a termination of the path like how symbolic execution tools treat unsolvable paths. In all, we try to avoid the false positive cases while finding more true positive cases.

Table 2 presents our experimental result. Neural network models including LSTM, Tree-LSTM, PCC can only find half of the unsolvable constraints. To achieve a better recall, they would inevitably misjudge more solvable constraints. The naive KNN cannot predict solvability when the distance between feature vectors is large, e.g., in the last two datasets. Furthermore, the adaptive approach (incremental KNN) performs better than other models, which achieve a stable recall and precision, with f1-score over 0.74. According to our observation, as the added data increases, the feature vectors that determine the classification result mostly come from the constraints of the same program, so our adaptive approach plays an important role in performance improvement.

In short, non-adaptive models can find some correlation between the solving time and constraint models, but only in a limited range. The prediction cannot achieve a promising result in both recall and precision at the same time. The adaptive approach performs better than non-adaptive models because the extended data gradually optimize the existing model.

5.2.2 Feature Analysis. We do specific research to analyze the reason why the neural network models have difficulty in time prediction for some programs. We use the odds ratio [17] to identify the influence of features on the solving time. The odds ratio δ_i for feature i is defined like this:

Table 2: The classification evaluation result of non-adaptive and adaptive approaches.

	GNU coreutils(angr)			Busybox utilities(angr)			SMT-COMP			GNU coreutils(KLEE)		
	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1	Precision	Recall	F1
KNN	0.552	0.422	0.478	0.77	0.747	0.758	0.375	0.193	0.254	0.198	0.244	0.219
Incremental KNN	0.737	0.75	0.743	0.768	0.834	0.800	0.792	0.751	0.771	0.769	0.808	0.788
LSTM	0.719	0.506	0.593	0.674	0.36	0.469	0.277	0.301	0.289	0.393	0.485	0.434
Tree-LSTM	0.634	0.554	0.591	0.749	0.749	0.749	0.216	0.760	0.336	0.582	0.445	0.504
PCC(baseline)	0.461	0.544	0.499	0.319	0.630	0.423	0.285	0.856	0.428	0.512	0.483	0.497

Table 3: The odds ratio of selected features. N.A. stands for the absence of a certain feature or that all constraints without the feature are solvable, All stands for that all constraint models contain this feature.

	GNU(angr)	Busybox	SMT-COMP	GNU(KLEE)
and	0.987	0	0.046	9.998
or	0.638	0.071	2.149	0.026
not	N.A.	0.127	1.927	N.A.
ite	0.569	0.191	0.365	7.954
concat	0.984	N.A.	0.857	4.749
extract	1.033	All	0.563	N.A.
bvult	2.743	N.A.	1.483	9.911
bvule	0.845	All	0.417	0
bvadd	0.581	N.A.	2.090	2.906
bvsub	N.A.	N.A.	0.093	4.499
bvmul	0.776	N.A.	5.889	9.430
bvurem	0.045	0	0.182	2.959
bvshl	0.539	N.A.	0.203	0.976
bvlshr	N.A.	N.A.	3.417	0.400
distinct	8.276	0	0.424	N.A.

$$\delta_i = \frac{W_p^i}{W_n^i} * \frac{O_n^i}{O_p^i} \quad (4)$$

For the symbols in Eq. 4, W stands for the number of constraints that contain this feature, while O stands for the number of constraints that do not contain this feature, where p means the constraint is unsolvable, and n means the constraint is solvable.

To elaborate, the odds ratio larger than one represents the positive relevance between the feature and the solving time. The larger the value is, the higher the correlation is. We choose some of the commonly used features to show their result. To explain, most of the N.A. in Busybox, except `bvult`, is because all constraints without the feature are solvable, so the divisor is zero. From table 3, we can see that most features' odds ratio varies dramatically for different datasets, reflecting that no selected feature can decide the hardness of solving.

5.2.3 Time Prediction Evaluation. To better compare non-adaptive models, we also forecast the actual solving time instead of setting a threshold to classify the result. Our KNN model is not designed for regression tasks, so the models are limited to LSTM, Tree-LSTM,

Table 4: The regression evaluation result of predicted solving time.

	MSE			
	GNU(angr)	Busybox	SMT-COMP	GNU(KLEE)
LSTM	3791	1805	2267	2850
Tree-LSTM	4298	1915	2053	2963
PCC	4760	3301	3017	3689

and PCC (based on DNN). The predicted solving time ranges from zero to 300 seconds, and we evaluate it with MSE (Mean Squared Error) [16, 52]. Table 4 shows the MSE of predicted solving time, ranging from 1,800 to 5,000, which is an average result of multiple programs with very different results. Similar to solvability classification, the non-adaptive models do not work well for some new programs. The predicted time for all constraint models of these programs is close, which greatly affects the average MSE.

As for the model selection, no model stands out. Our models perform slightly better because our feature vectors are separated by each assertion and also contain the information of symbolic variables compared to baseline. Even better, our feature is easier to obtain since we do not reconstruct the SMT logic tree. The Tree-LSTM structure does not bring obvious improvement despite its heavy cost for training and prediction. Besides, there is an interesting finding which enlightens us to adopt the adaptive approach. When we separate the data with proportion instead of programs, the time prediction result is much better.

In short, we conjecture that machine learning techniques can find the correlation between the solving time and constraint models. The adaptive approach performs better than non-adaptive models due to the unknown patterns in the constraints of some new programs.

5.3 Simulation Experiment

To evaluate the effectiveness of our approach and answer RQ3, we conduct simulation experiments with several single programs. To elaborate, we calculate the solving time with Eq.(2) described in Section 3.3. Our approach only applies the classifier to constraint models that take more than one second (t_1). The constraints whose predicted time lower than the threshold t_2 would be summed into our simulation time; other constraints with predicted time greater than the threshold t_2 would be directly skipped.

We show the distribution of SMT queries for eight programs in Table 5. The constraint models of six programs generated by angr

Table 5: Time distribution of SMT queries for eight simulation programs. Six of them are collected with symbolic execution tools. Two are from the benchmark SMT-COMP dataset.

Program	Target Binary		
	Dataset	Timeout Query	Test Query
sha1sum	GNU(angr)	315	1,366
tail	GNU(angr)	97	1,115
ifconfig	Busybox(angr)	18	433
klogd	Busybox(angr)	16	408
mcm	SMT-COMP	91	122
lfsr	SMT-COMP	44	240
stat	GNU(KLEE)	156	1,925
cp	GNU(KLEE)	189	1,706

and KLEE follow the actual order of BFS symbolic execution. For constraint models of the other two programs from SMT-COMP, we directly adopt their original orders in the dataset. Therefore, the portion of timeout queries of the two programs is larger than other programs. Employing programs with different solving time distributions can help us better understand the performance of our system.

Table 6 shows the time of original solving and after using the predictor. We can observe that the time gets improved for all programs except lfsr. The result implies the time saved by our predictor can compensate for the prediction cost. Moreover, it reflects the importance of the first solving phase that filters out easy-solving constraints and one second should be a suitable threshold for most programs. Based on the result, we conjecture that solving time prediction could effectively accelerate solving efficiency at the cost of some misjudgment.

The results of LSTM and PCC demonstrate that the non-adaptive model fails to apply the same threshold of the training program to a few testing programs. The lfsr is an example of applying a too high threshold so that all constraints are treated solvable. The prediction even brings in a negative effect. The stat with LSTM is an example of adopting a too low threshold that most constraints are treated unsolvable. The predictor brings in too many false positive cases, abandoning about one-third of the queries which trigger the predictor. The result would mislead the path selection of symbolic execution.

In comparison, the adaptive approach incremental KNN finds most timeout cases for most programs with the least errors. The prediction result is consistent with the previous classification experiments, which shows that our data selection would not affect the vector similarity in KNN when the model increases more negative cases. In general, The adaptive KNN speeds up the solving by 1.25x (18492/14224) to 3x (117907/37831) times according to the distribution of timeout constraints.

We do not pursue the least solving time since it may result in symbolic execution effect degradation. There is a trade-off between the improvement rate and the time threshold. With t_2 decreasing, symbolic execution saves more time but ignores some of the executable paths. So, you may change the time threshold to control the

Table 6: Results of our simulation experiment with eight programs. For each program, we compare the time required for constraint solving with three predictors. Original solving shows the solving time spent by original symbolic execution and the number of timeout queries involved.

		time	TP	FP	FN
sha1sum	Original Solving	117,907	315	-	-
	Incremental KNN	37,831	251	101	64
	LSTM	64,255	165	114	150
	PCC	29,070	230	375	85
tail	Original Solving	57,177	97	-	-
	Incremental KNN	38,146	63	15	34
	LSTM	40,026	53	31	44
	PCC	36,996	55	58	42
ifconfig	Original Solving	18,492	18	-	-
	Incremental KNN	14,224	16	2	2
	LSTM	17,789	4	0	14
	PCC	13,229	16	30	2
klogd	Original Solving	14,211	16	-	-
	Incremental KNN	10,649	13	2	3
	LSTM	11,938	8	7	8
	PCC	11,677	8	15	8
mcm	Original Solving	28,113	91	-	-
	Incremental KNN	6,070	74	1	17
	LSTM	190	91	17	0
	PCC	11,062	57	2	34
lfsr	Original Solving	18,793	44	-	-
	Incremental KNN	10,577	27	5	17
	LSTM	19,049	0	0	44
	PCC	19,067	0	0	44
stat	Original Solving	69,506	156	-	-
	Incremental KNN	33,821	123	87	33
	LSTM	31,052	100	718	56
	PCC	54,449	51	179	105
cp	Original Solving	71,523	189	-	-
	Incremental KNN	25,857	157	34	32
	LSTM	30,255	126	133	63
	PCC	16,649	167	207	22

path exploration of symbolic execution. If the analysis requires high coverage of executable paths, you should take a higher threshold. On the other hand, if you want a fast exploration of program paths with symbolic execution, you can choose a lower threshold to avoid difficult solving even if they are solvable by spending a long time.

In general, the adaptive solving time prediction increases the solving speed of constraint models by 1.25x to 3x times at the cost of a few more unexplored paths in symbolic execution.

6 THREATS TO VALIDITY

The solving time precision and dataset selection method would affect the prediction effectiveness. Now we discuss these two threats to validity and how we alleviate such threats.

The solving time of a constraint model is unstable when the running context changes. The most significant factor is whether

with symbolic execution or not. Most constraint models require less time to find a solution when conducting pure solving. Labeling these data differently would introduce a huge difference. To alleviate this threat, we purely query constraints with the same setting to adjust solving time. We have also experimented on different timeout settings to avoid the bad time threshold choosing affect the classification result. In all experiments, the adaptive approach outperforms non-adaptive approaches and keeps a decent result in prediction and the solving time optimization problem.

To balance the data of different labels, we use the under-sampling to choose the training data, which means it is not the actual solving order in the symbolic execution process. The real data is diluted by more negative cases, which would affect the model result, especially for incremental KNN that heavily depends on data. So in the simulation test, we apply our system to all constraint models of the program just like the actual execution. There is little degradation of the precision and overall we can see improvements in solving efficiency for most programs we have tested.

7 RELATED WORK

This paper focuses on boosting symbolic execution via predicting constraint solving time. There is also some existing work in improving SMT solving efficiency or symbolic execution. Next, we compare our work with related papers in these two fields.

7.1 SMT solving

Constraint solving is a key issue in software analysis since formula verification is widely used in automated program analysis, testing [2, 5, 13]. However, reasoning with certain theories is still hard to handle, such as floating-point numbers, non-linear arithmetic. Researchers try to simplify the formula and optimize solution search. England et al. use cylindrical algebraic decomposition to simplify non-linear real arithmetic [15]. SymFPU [7] optimizes the bit blasting to solve floating-point symbolic variables. JFS [29] uses a coverage-guided fuzzer to search the inputs for floating-point constraints. Yao et al. utilize the word-level information to optimizes the bit blast of bit-vectors. These methods are all based on the reasoning theories requiring professional knowledge.

With neural networks becoming a new trend, many temptations are made to use them in constraint solving. Luo et al. build a Seq-to-Seq neural network to solve the SAT (Satisfiability) question [32]. Wen et al. [50] transform the constraint model into a 2-dimension matrix and use a deep neural network to classify satisfiability. These approaches avoid the expensive solving but cannot generate solutions to concretize the symbolic variable for SMT formulas.

The work most similar to ours is our baseline [51]. It proposes a meta-solver that enables symbolic execution tools to choose the best solver for different constraint models. The meta-solver employs a path constraint classifier (PCC) to handle SMT scripts over well-defined features. Both works optimize SMT solve process for symbolic execution tools. Unlike PCC, we focus on solving time prediction for a single solver. Moreover, we propose a practical way to integrate our prediction module into symbolic execution tools.

7.2 Symbolic Execution

Symbolic execution tools usually suffer from path explosion and hard constraint solving. To alleviate path explosion, symbolic execution tools employ different heuristic search methods to select the most valuable paths [9]. For example, KLEE [10] provides both random and coverage-optimized search to cover new code. Sage [21] introduces a generational search. Similarly, we can assign different priorities to the path with our predicted constraint solving time. Other solutions of path explosion include path pruning [6, 8], path merging [27, 41, 45] and concretization [22, 35]. They try to reduce the number of paths with consistency in attributes such as constraints or the value of symbolic variables. Specifically, The path merging would bring the extra burden to constraint solving since it generates more complex constraint models, which may cause more unsolvable constraints within the time limit.

Some symbolic execution systems have optimization strategies to preprocess constraints before the query. They reuse constraint solutions to avoid redundant constraint solving during symbolic execution. For example, KLEE uses the counter-example cache to eliminate queries. Green [25, 47] stores constraint and solutions information to reuse it across programs. We propose the adaptive approach to use the similarity of sequential constraint models to predict solving time. These works are orthogonal to our approach, and our approach can be applied together with those optimization strategies as long as there are constraint models that cannot be solved efficiently.

8 CONCLUSION

This paper studies the efficiency issues of symbolic execution with respect to constraint solving. We have designed a system SMTimer that predicts the required solving time for a constraint model and helps symbolic execution tools escape the unavailing constraint solving process in advance. Especially, SMTimer employs an adaptive model to utilize the similarity of constraint. During runtime, the predictor is triggered only if the time spent for solving the current model has exceeded a relatively small threshold. When triggered, the adaptive approach predicts the solvability and updates the model with ground truth to classify the result better. We have conducted evaluation experiments based on thousands of real-world constraint models collected from angr and KLEE. Experimental results show that our adaptive approach has achieved an F1-score of 0.74, which is better than non-adaptive models. Furthermore, we have conducted a simulation experiment that shows the adaptive approach can improve the efficiency of constraint solving by 1.25x to 3x. In conclusion, our system can effectively boost the constraint solving process and help symbolic execution analyze more paths within a limited time.

ACKNOWLEDGMENTS

This work was supported by CERNET Research Grant NGII20190410 and the National Natural Science Foundation of China (Project No. 61971145).

REFERENCES

- [1] D. Avola, L. Cinque, G. L. Foresti, F. Lamacchia, M. R. Marini, L. Perini, K. Qorraj, and G. Telesca. 2019. A Shape Comparison Reinforcement Method Based on

- Feature Extractors and F1-Score. In *2019 IEEE International Conference on Systems, Man and Cybernetics (SMC)*. 2155–2159. <https://doi.org/10.1109/SMC.2019.8914601>
- [2] Sébastien Bardin, Nikolaj Bjørner, and Cristian Cadar. 2019. Bringing CP, SAT and SMT together: Next Challenges in Constraint Solving (Dagstuhl Seminar 19062). *Dagstuhl Reports* 9, 2 (2019), 27–47. <https://doi.org/10.4230/DagRep.9.2.27>
 - [3] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. 2016. The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org.
 - [4] Clark Barrett, Aaron Stump, Cesare Tinelli, et al. 2010. The smt-lib standard: Version 2.0. In *Proceedings of the 8th international workshop on satisfiability modulo theories*, Vol. 13. 14.
 - [5] Clark W. Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. 2009. Satisfiability Modulo Theories. In *Handbook of Satisfiability*. 825–885. <https://doi.org/10.3233/978-1-58603-929-5-825>
 - [6] Peter Boonstoppel, Cristian Cadar, and Dawson R. Engler. 2008. RWset: Attacking Path Explosion in Constraint-Based Test Generation. In *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference (TACAS)*. 351–366. https://doi.org/10.1007/978-3-540-78800-3_27
 - [7] Martin Brain, Florian Schanda, and Yucheng Sun. 2019. Building Better Bit-Blasting for Floating-Point Problems. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems TACAS*. 79–98. https://doi.org/10.1007/978-3-030-17462-0_5
 - [8] Suhabe Bugrara and Dawson R. Engler. 2013. Redundant State Detection for Dynamic Symbolic Execution. In *2013 USENIX Annual Technical Conference*. USENIX Association, 199–211. <https://www.usenix.org/conference/atc13/technical-sessions/presentation/bugrara>
 - [9] Jacob Burnim and Koushik Sen. 2008. Heuristics for Scalable Dynamic Test Generation. In *23rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 443–446. <https://doi.org/10.1109/ASE.2008.69>
 - [10] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs.. In *OSDI*, Vol. 8. 209–224.
 - [11] Lori A. Clarke. 1976. A program testing system. In *Proceedings of the 1976 Annual Conference*. 488–491. <https://doi.org/10.1145/800191.805647>
 - [12] Leonardo Mendonça de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
 - [13] Leonardo Mendonça de Moura and Nikolaj Bjørner. 2011. Satisfiability modulo theories: introduction and applications. *Commun. ACM* 54, 9 (2011), 69–77. <https://doi.org/10.1145/1995376.1995394>
 - [14] John Duchi, Elad Hazan, and Yoram Singer. 2011. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of machine learning research* 12, Jul (2011), 2121–2159.
 - [15] Matthew England, Russell J. Bradford, and James H. Davenport. 2019. Cylindrical Algebraic Decomposition with Equational Constraints. *CoRR* abs/1903.08999 (2019). [arXiv:1903.08999](https://arxiv.org/abs/1903.08999) <http://arxiv.org/abs/1903.08999>
 - [16] Yariv Ephraim and David Malah. 1984. Speech enhancement using a minimum-mean square error short-time spectral amplitude estimator. *IEEE Transactions on acoustics, speech, and signal processing* 32, 6 (1984), 1109–1121.
 - [17] George Forman et al. 2003. An extensive empirical study of feature selection metrics for text classification. *J. Mach. Learn. Res.* 3, Mar (2003), 1289–1305.
 - [18] Kilian Förster, Samuel Monteleone, Alberto Calatroni, Daniel Roggen, and Gerhard Troster. 2010. Incremental kNN classifier exploiting correct-error teacher for activity recognition. In *2010 ninth international conference on machine learning and applications*. 445–450.
 - [19] Marco Gario and Andrea Micheli. 2015. PySMT: a solver-agnostic library for fast prototyping of SMT-based algorithms. In *SMT Workshop 2015*.
 - [20] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. 213–223. <https://doi.org/10.1145/1065010.1065036>
 - [21] Patrice Godefroid, Michael Y. Levin, and David A. Molnar. 2008. Automated White-box Fuzz Testing. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. <https://www.ndss-symposium.org/ndss2008/automated-whitebox-fuzz-testing/>
 - [22] Fabio Gritti, Lorenzo Fontana, Eric Gustafson, Fabio Pagani, Andrea Continella, Christopher Kruegel, and Giovanni Vigna. 2020. SYMBION: Interleaving Symbolic with Concrete Execution. In *Proceedings of the IEEE Conference on Communications and Network Security (CNS)*.
 - [23] Jun Gu, Paul W Purdom, John Franco, and Benjamin W Wah. 1996. *Algorithms for the satisfiability (SAT) problem: A survey*. Technical Report. Cincinnati Univ oh Dept of Electrical and Computer Engineering.
 - [24] Steven CH Hoi, Jialei Wang, and Peilin Zhao. 2014. Libol: A library for online learning algorithms. *Journal of Machine Learning Research* 15, 1 (2014), 495.
 - [25] Xiangyang Jia, Carlo Ghezzi, and Shi Ying. 2015. Enhancing reuse of constraint solutions to improve symbolic execution. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*. 177–187. <https://doi.org/10.1145/2771783.2771806>
 - [26] James C. King. 1976. Symbolic Execution and Program Testing. *Commun. ACM* 19, 7 (1976), 385–394. <https://doi.org/10.1145/360248.360252>
 - [27] Volodymyr Kuznetsov, Johannes Kinder, Stefan Bucur, and George Candea. 2012. Efficient state merging in symbolic execution. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 193–204. <https://doi.org/10.1145/2254064.2254088>
 - [28] Kiran Lakhota, Nikolai Tillmann, Mark Harman, and Jonathan De Halleux. 2010. Flopsy-search-based floating point constraint solving for symbolic execution. In *IFIP International Conference on Testing Software and Systems*. 142–157.
 - [29] Daniel Liew, Cristian Cadar, Alastair F. Donaldson, and J. Ryan Stinnett. 2019. Just fuzz it: solving floating-point constraints using coverage-guided fuzzing. In *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/SIGSOFT FSE)*. 521–532. <https://doi.org/10.1145/3338906.3338921>
 - [30] Daniel Liew, Daniel Schemmel, Cristian Cadar, Alastair F. Donaldson, Rafael Zähl, and Klaus Wehrle. 2017. Floating-point symbolic execution: a case study in n-version programming. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 601–612. <https://doi.org/10.1109/ASE.2017.8115670>
 - [31] Peng Liu, Xiangyu Zhang, Marco Pistoia, Yunhui Zheng, Manoel Marques, and Lingfei Zeng. 2017. Automatic text input generation for mobile testing. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. IEEE, 643–653. <https://doi.org/10.1109/ICSE.2017.65>
 - [32] Xuefeng Luo, Alina Baranova, and Jonas Biegert. 2019. ProblemSolver at SemEval-2019 Task 10: Sequence-to-Sequence Learning and Expression Trees. In *Proceedings of the 13th International Workshop on Semantic Evaluation*. 1292–1296. <https://doi.org/10.18653/v1/s19-2227>
 - [33] Mark Mossberg, Felipe Manzano, Eric Hennenfent, Alex Groce, Gustavo Grieco, Josselin Feist, Trent Brunson, and Artem Dinaburg. 2019. Manticore: A user-friendly symbolic execution framework for binaries and smart contracts. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 1186–1189.
 - [34] Hristina Palikareva and Cristian Cadar. 2013. Multi-solver Support in Symbolic Execution. In *Computer Aided Verification - 25th International Conference (CAV) Saint Petersburg, Russia, July 13-19, 2013. Proceedings*. 53–68. https://doi.org/10.1007/978-3-642-39799-8_3
 - [35] Awanish Pandey, Phani Raj Goutham Kotcharlakota, and Subhajit Roy. 2019. Deferred concretization in symbolic execution via fuzzing. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 228–238.
 - [36] David M Perry, Andrea Mattavelli, Xiangyu Zhang, and Cristian Cadar. 2017. Accelerating array constraints in symbolic execution. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 68–78.
 - [37] Xiao Qu and Brian Robinson. 2011. A Case Study of Concolic Testing Tools and their Limitations. In *Proceedings of the 5th International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 117–126. <https://doi.org/10.1109/ESEM.2011.20>
 - [38] Minghui Quan. 2016. Hotspot symbolic execution of floating-point programs. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 1112–1114. <https://doi.org/10.1145/2950290.2983966>
 - [39] Guozheng Rao, Weihang Huang, Zhiyong Feng, and Qiong Cong. 2018. LSTM with sentence representations for document-level sentiment classification. *Neurocomputing* 308 (2018), 49–57.
 - [40] Koushik Sen and Gul Agha. 2006. CUTE and jCUTE: Concolic Unit Testing and Explicit Path Model-Checking Tools. In *International Conference on Computer Aided Verification (CAV)*. 419–423. https://doi.org/10.1007/11817963_38
 - [41] Koushik Sen, George C. Necula, Liang Gong, and Wontae Choi. 2015. MultiSE: multi-path symbolic execution using value summaries. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 842–853. <https://doi.org/10.1145/2786805.2786830>
 - [42] Yan Shoshitaishvili, Ruoyu Wang, Christopher Salls, Nick Stephens, Mario Polino, Andrew Dutcher, John Groen, Siji Feng, Christophe Hauser, Christopher Kruegel, et al. 2016. Sok(state of) the art of war: Offensive techniques in binary analysis. In *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 138–157.
 - [43] Kai Sheng Tai, Richard Socher, and Christopher D. Manning. 2015. Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing (ACL)*. 1556–1566. <https://doi.org/10.3115/v1/p15-1150>
 - [44] Y. Teshima and Y. Watanabe. 2018. Bug Detection Based on LSTM Networks and Solution Codes. In *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 3541–3546. <https://doi.org/10.1109/SMC.2018.00599>
 - [45] David Trubish, Andrea Mattavelli, Noam Rinetzk, and Cristian Cadar. 2018. Chopped symbolic execution. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. 350–360. <https://doi.org/10.1145/3180155.3180251>

- [46] Valerie Vaquet and Barbara Hammer. 2020. Balanced SAM-kNN: Online Learning with Heterogeneous Drift and Imbalanced Data. In *International Conference on Artificial Neural Networks*. 850–862.
- [47] Willem Visser, Jaco Geldenhuys, and Matthew B. Dwyer. 2012. Green: reducing, reusing and recycling constraints in program analysis. In *Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering*. 58. <https://doi.org/10.1145/2393596.2393665>
- [48] Lei Wang, Yan Wang, Deng Cai, Dongxiang Zhang, and Xiaojiang Liu. 2018. Translating a math word problem to an expression tree. *arXiv preprint arXiv:1811.05632* (2018).
- [49] Tjark Weber, Sylvain Conchon, David Déharbe, Matthias Heizmann, Aina Niemetz, and Giles Reger. 2019. The SMT Competition 2015-2018. *J. Satisf. Boolean Model. Comput.* 11, 1 (2019), 221–259. <https://doi.org/10.3233/SAT190123>
- [50] Junye Wen, Mujahid Khan, Meiru Che, Yan Yan, and Guowei Yang. 2020. Constraint Solving with Deep Learning for Symbolic Execution. *CoRR* abs/2003.08350 (2020). [arXiv:2003.08350](https://arxiv.org/abs/2003.08350) <https://arxiv.org/abs/2003.08350>
- [51] Sheng-Han Wen, Wei-Loon Mow, Wei-Ning Chen, Chien-Yuan Wang, and Hsu-Chun Hsiao. 2019. Enhancing symbolic execution by machine learning based solver selection. In *Workshop on Binary Analysis Research (BAR)*, Vol. 2.
- [52] Cort J Willmott and Kenji Matsuura. 2005. Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate research* 30, 1 (2005), 79–82.
- [53] Hui Xu, Zirui Zhao, Yangfan Zhou, and Michael R Lyu. 2018. Benchmarking the capability of symbolic execution tools with logic bombs. *IEEE Transactions on Dependable and Secure Computing* 17, 6 (2018), 1243–1256.
- [54] Bianca Zadrozny. 2004. Learning and evaluating classifiers under sample selection bias. In *Proceedings of the twenty-first international conference on Machine learning*. 114. <https://doi.org/10.1145/1015330.1015425>